

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Иркутский национальный исследовательский технический
университет»
Институт информационных технологий и анализа данных

О Т Ч Ё Т

о прохождении _____ учебной практики
(вид практики: учебная/производственная)

технологической (проектно-технологической) практики
(тип практики: технологическая/научно-исследовательская работа/преддипломная и др.)

В _____ ИРНITU
(наименование профильной организации)



<https://irkutsk.superjob.ru/resume/python-developer-56829970.html>



https://irkutsk.hh.ru/resume/b4134439ff112e1a9b0039ed1f4b505973435a?hhtmFrom=applicant_profile

Обучающегося [Токарев М.К., ИСИ6-25-1]
(ФИО, группа, подпись)

Руководитель практики от института ИТиАД
[Кононенко Р.В., доцент института ИТиАД]
(ФИО, должность, подпись)

Руководитель образовательной
программы
Кононенко Р.В., доцент института ИТиАД
(ФИО, должность, подпись)

Оценка по практике _____
(ФИО, подпись, дата)

Содержание отчета на [_____]стр.
Приложение к отчету на _____ стр.

Иркутск 20[26]

Индивидуальное задание на прохождение
учебной практики: технологической (проектно-технологической)
практики

для Токарева Максима Константиновича

(ФИО обучающегося полностью)

обучающегося 1 курса группы ИСИ6-25-1
по направлению подготовки Информационные системы и технологии
профиль Интеллектуальные системы обработки информации и управления

Место прохождения практики: ИРНИТУ

Сроки прохождения практики с «15» июня 2026 г. по «28» июня 2026 г.

Цели и задачи прохождения практики: [Закрепление и углубление теоретических знаний в области алгоритмизации, программирования на языках C++ и Python, проектирования и программирования микроконтроллерных систем на базе Arduino, а также освоение базовых методов компьютерного зрения.]

Содержание практики, вопросы, подлежащие изучению: [Углубление навыков написания кода на C++; Освоение принципов проектирования схем на Arduino и методы компьютерного зрения в OpenCV на Python.]

Планируемые результаты практики: [Освоение методов разработки и оптимизации алгоритмов на C++, практических навыков подключения и программирования периферии Arduino, а также инструментов цифровой обработки изображений]

Руководитель практики от
института ИТиАД

_____/Кононенко Р.В./

(подпись)

Согласовано:

Руководитель ООП

_____/Кононенко Р.В./

(подпись)

« » 2026 г.

С настоящим индивидуальным заданием и с программой практики
ознакомлен(а), задание принято к исполнению

«15» июня 2026 г.

(подпись)

Содержание

Введение.....	4
Задание №1.....	5
Задание №2.....	7
Задание №3.....	8
Задание №4.....	10
Задание №5.....	13
Задание №6.....	16
Задание №7.....	18
Задание №8.....	20
Задание №9.....	24
Задание №10.....	27
Задание №11.....	30
Заключение.....	33
Список литературы.....	34

Введение

С 15 по 28 июня 2026 года на базе ФГБОУ ВО «ИРНИТУ» проходила учебная практика, направленная на систематизацию теоретических знаний по профильным дисциплинам и формирование навыков их практического применения.

Программа практики включала выполнение 11 практических заданий по трем ключевым блокам: алгоритмизации и разработке на C++, проектированию устройств на базе микроконтроллеров Arduino, а также применению методов компьютерного зрения на Python с использованием библиотеки OpenCV.

По итогам практики удалось успешно отработать теоретическую базу, углубить профессиональные компетенции и приобрести актуальный прикладной опыт.

Задание №1

Незнайка в своей экспедиции на Луну оказался на вершине лунной горы. Спуск вниз опасен, поэтому он взял с собой карту склона горы, где числами обозначено, сколько минут требуется на этот участок маршрута. Спуск происходит сверху вниз на один из соседних участков. Пример наиболее короткого маршрута выделен красным цветом, сумма чисел = 10 (см. рисунок 1).

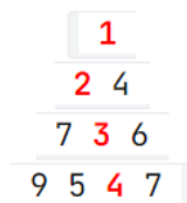


Рисунок 1

Напишите программу, рассчитывающую минимальное время спуска (сумму чисел в пути с вершины до основания). Пример входных и выходных данных представлен на рисунке 2.

Sample Input:

```
13
88
43 56
34 49 67
69 68 52 54
41 22 44 64 50
55 12 59 48 30 85
82 85 98 39 93 32 61
45 17 51 96 18 77 47 16
85 85 75 93 73 65 14 53 57
10 33 47 66 95 49 92 95 73 94
55 81 88 62 70 61 63 47 81 91 28
19 24 75 21 46 74 28 42 60 52 84 52
16 77 18 30 30 14 64 38 50 62 30 40 96
```

Sample Output:

```
604
88 43 34 68 22 12 85 17 75 47 62 21 30
```

Рисунок 2

Алгоритм программы:

- 1) Программа принимает высоту пирамиды, с помощью цикла заполняет список списков с значениями вершин
- 2) Циклом for перебираем строки, начиная с предпоследней до первой
- 3) Выбор минимального соседа - для элемента `heap[i][j]` находятся два его возможных продолжения на строке ниже — это `heap[i + 1][j]` (левый сосед) и `heap[i + 1][j + 1]` (правый сосед). Из них выбирается минимальное значение;
- 4) К текущему элементу прибавляется минимальная сумма спуска ниже;
- 5) Результат: По завершении циклов в элементе `heap[0][0]` оказывается минимальная сумма всего пути от вершины до основания, которая выводится на экран `print(heap[0][0])`;
- 6) Циклом for начиная со второй строки ищем путь в новой пирамиде, ища минимальный элемент среди возможных путей;

Код программы представлен на рисунке 3.

```
heap = []
height = int(input())
for i in range(height):
    heap.append([int(x) for x in input().split()])
for i in range(height - 2, -1, -1):
    for j in range(len(heap[i])):
        min_vertex = min(heap[i + 1][j], heap[i + 1][j + 1])
        heap[i][j] += min_vertex
print(heap[0][0])
# print(heap)
ind = 0
vertex_ = heap[0][0]
for i in range(1, height):
    if heap[i][ind] < heap[i][ind + 1]:
        print(vertex_ - heap[i][ind], end=" ")
        vertex_ = heap[i][ind]
    else:
        print(vertex_ - heap[i][ind + 1], end=" ")
        ind += 1
        vertex_ = heap[i][ind]
print(vertex_)
```

Рисунок 3

Задание №2

После метеоритной атаки компьютерная сеть для управления лунными заводами разбилась на части, нужно объединить её в единое целое. Каждый фрагмент сети представлен в виде ненаправленного графа.

Вам известно общее число вершин графа (узлы сети, не более 1000) и набор рёбер (сохранившиеся линии связи, не более 1000).

Определите, какое минимальное число линий связи нужно дополнительно построить, чтобы сеть стала единой.

Пример входных и выходных данных представлен на рисунке 4.

Sample Input:

```
10 6
1 2
2 8
4 10
5 9
6 10
7 9
```

Sample Output:

```
3
```

Рисунок 4

Алгоритм программы:

- 1) Программа считывает количество число ребер и вершин, создает пустой список смежности
- 2) Циклом заполняем ненаправленный граф (список смежности)
Код программы представлен на рисунке 5.
- 3) Ищем не помеченные вершины (компоненты связности) и запускаем обход в ширину с этой вершины:
3.1) При помощи обхода в ширину (dfs): в массиве visited помечаем посещенный вершины, для не посещенных соседних вершин рекурсивно вызываем dfs
- 4) Посчитав общее количество изолированных групп components_count, вычитаем единицу, так как для соединения k компонентов в одну цепочку требуется k - 1 новых рёбер

```

n, m = map(int, input().split())
graph = [[] for _ in range(n + 1)]
for _ in range(m):
    u, v = map(int, input().split())
    graph[u].append(v)
    graph[v].append(u)
visited = [False] * (n + 1)
def dfs(v): 2+ usages
    visited[v] = True
    for neighbor in graph[v]:
        if not visited[neighbor]:
            dfs(neighbor)
components_count = 0
for i in range(1, n + 1):
    if not visited[i]:
        components_count += 1
        dfs(i)
print(components_count - 1)

```

Рисунок 5

Задание №3

Вам поручено возглавить отдел разведки снежной ситуации и быть способным чрезвычайно быстро отвечать на запросы центра. Сам город состоит из нескольких, расположенных подряд, улиц, каждая из которых абсолютно похожа на любую другую.

- информация о снеге передается вам в виде тройки чисел – 1 в качестве идентификатора события, уникального индекса улицы и количество миллиметров выпавшего снега.

- запросы в свою очередь так же имеют вид тройки чисел – 2 в качестве идентификатора события, индекс улицы с которой нужно суммировать количество выпавшего снега и индекс улицы по которую нужно суммировать, крайние улицы должны быть включены.

На каждый запрос второго типа надо вывести единственное число – суммарное выпавшего на них снега с момента начала наблюдения.

Пример входных и выходных данных представлен на рисунке 6.

Sample Input 1:

```
6 5
2 1 6
1 3 2
2 2 4
1 6 3
2 1 6
```

Sample Input 2:

```
5 3
1 3 7
1 1 4
2 1 5
```

Sample Output 1:

```
0
2
5
```

Sample Output 2:

```
11
```

Рисунок 6

Алгоритм программы:

решение задачи об учете снега на улицах с помощью структуры данных Дерево Фенвика:

- 1) Класс FenwickTree, хранящий количество N улиц (size), а также вектор tree размером $N + 1$, по умолчанию заполненный нулями.
- 2) В функции add добавление снега к улице с номером idx
- 3) В функции query считывается сумма выпавшего снега от первой улицы до улицы idx включительно
- 4) Функция queryRange считает сумму снега на отрезке от left до right включительно. Берём сумму от начала до конца отрезка и вычитаем всё, что находится левее границы left
- 5) В функции main:
 - 5.1) считывается количество улиц n и количество запрос q ;
 - 5.2) Создаётся объект ft класса FenwickTree;
 - 5.3) В цикле последовательно считываем тип события type:

Тип 1 (Выпадение снега): считываем индекс улицы и количество снега, затем обновляем структуру через ft.add()

Тип 2 (Запрос центра): считываем диапазон left и right, затем мгновенно вычисляем и выводим сумму через ft.queryRange().

Символ "\n" используется вместо endl, так как endl принудительно скидывает буфер, что медленнее.

```

#include <vector>

using namespace std;

class FenwickTree {
private:
    int size;
    vector<long long> tree;

public:
    FenwickTree(int n) : size(n), tree(n + 1, 0) {}
    void add(int idx, long long delta) {
        for (; idx <= size; idx += idx & -idx) {
            tree[idx] += delta;
        }
    }
    long long query(int idx) {
        long long sum = 0;
        for (; idx > 0; idx -= idx & -idx) {
            sum += tree[idx];
        }
        return sum;
    }
    long long queryRange(int left, int right) {
        if (left > right) swap(left, right);
        return query(right) - query(left - 1);
    }
};

int main() {
    int n, q;
    if (!(cin >> n >> q)) return 0;
    FenwickTree ft(n);
    for (int i = 0; i < q; ++i) {
        int type;
        cin >> type;
        if (type == 1) {
            int street_idx;
            long long val;
            cin >> street_idx >> val;
            ft.add(street_idx, val);
        } else if (type == 2) {
            int left, right;
            cin >> left >> right;
            cout << ft.queryRange(left, right) << "\n";
        }
    }
    return 0;
}

```

Рисунок 7

Задание №4

Перестановка P длины n – это упорядоченный набор, содержащий числа от 1 до n , каждое из которых входит в него ровно один раз. Например, перестановкой длины 13 является набор (5 11 13 12 6 1 8 4 10 9 7 2 3). Само

название говорит о том, для чего предназначен этот объект. Например, можно при помощи перестановки букв зашифровать слово. Для примера возьмем приведенную выше перестановку и слово transposition, которое состоит тоже из 13 букв. Далее, следуя перестановке, на первую позицию поставим пятую букву слова, на вторую – одиннадцатую букву и так далее. В итоге получим sinoptsntiora. К этому слову снова применим эту же перестановку и получим poartsnoitsin. Повторив эти стадии шифрования k раз, получим зашифрованное сообщение (см. рисунок 8).

t	r	a	n	s	p	o	s	i	t	i	o	n
1	2	3	4	5	6	7	8	9	10	11	12	13
5	11	13	12	6	1	8	4	10	9	7	2	3
s	i	n	o	p	t	s	n	t	i	o	r	a
1	2	3	4	5	6	7	8	9	10	11	12	13
5	11	13	12	6	1	8	4	10	9	7	2	3
p	o	a	r	t	s	n	o	i	t	s	i	n

Рисунок 8

Вам дано зашифрованное таким образом слово, шифрующая перестановка P и число k. Необходимо восстановить слово. Пример входных и выходных данных представлен на рисунке 9.

Sample Input:

```
13 2
5 11 13 12 6 1 8 4 10 9 7 2 3
poartsnoitsin
```

Sample Output:

```
transposition
```

Рисунок 9

Алгоритм программы:

1) Вводятся два числа: n (длина строки) и k (количество примененных операций/сдвигов). Считывается перестановка p. Массив преобразуется к 0-индексации (int(x) - 1) ;

- 2) Запускается внешний цикл `for`, который повторяется `l` раз, чтобы "отмотать" изменения назад; На каждом шаге создается новый пустой массив `previous_word` длины `n`
 - 3) перебор всех индексов от 0 до `n - 1` для нахождения независимых циклов перестановки;
 - 4) С помощью внутреннего цикла по индексам `i` от 0 до `n - 1` происходит восстановление символов на предыдущем шаге:
 - 5) Значение из текущей строки `res[i]` помещается в позицию `p[i]` нового массива (`previous_word[p[i]] = res[i]`)
- Код программы представлен на рисунке 10.

```
def solve():
    n, k = map(int, input().split())
    p = [int(x) - 1 for x in input().split()]
    word = input().strip()
    res = list(word)
    for _ in range(k):
        previous_word = [''] * n
        for i in range(n):
            previous_word[p[i]] = res[i]
        res = previous_word
    print(''.join(res))

solve()
```

Рисунок 10

Задание №5

Дана матрица, состоящая из 1 и 0. Значениями 1 в матрице нарисована некоторая фигура. Необходимо определить координаты верхнего левого и нижнего правого углов параллельного осям ограничивающего прямоугольника, т.е. такого прямоугольника, минимального размера, в который фигура помещается полностью и при этом ни одна точка исходной фигуры не попадает на стороны прямоугольника.

Пример входных и выходных данных представлен на рисунке 11.

Sample Input:

```
10 10
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 1 1 1 0 0 0 0
0 0 0 1 1 1 0 0 0 0
0 0 0 1 1 1 0 0 0 0
0 0 0 1 1 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
```

Sample Output:

```
3 2 8 6
```

Рисунок 11

Алгоритм программы:

- 1) Считывание размеров матрицы: программа запрашивает и считывает количество строк h и столбцов w . Если ввод не удался, завершает работу.
- 2) Инициализация границ фигуры: создаются переменные для поиска крайних координат единиц — минимальной/максимальной строки (min_row , max_row) и минимального/максимального столбца (min_col , max_col). Изначально минимальным границам присваиваются заведомо большие значения (h и w), а максимальным -1 .
- 3) Поэлементный обход матрицы: с помощью двух вложенных циклов программа проходит по каждой строке r (от 0 до $h-1$) и каждому столбцу c (от 0 до $w-1$), считывая значение элемента val .
- 4) Поиск минимального описывающего прямоугольника: если считанный элемент равен 1, то обновлениями проверяется и

фиксируется, является ли текущая клетка с координатами (r, c) новой самой верхней, нижней, левой или правой точкой фигуры.

- 5) Расширение рамок на 1 клетку (согласно условию задачи): так как по условию ни одна точка фигуры не должна попадать на стороны прямоугольника, границы расширяются:
 - 6) Верхний левый угол рамки: $top_left_row = min_row - 1$, $top_left_col = min_col - 1$;
 - 7) Нижний правый угол рамки: $bottom_right_row = max_row + 1$, $bottom_right_col = max_col + 1$.
 - 8) Вывод результата: программа выводит четыре полученных индекса (координаты верхнего левого и нижнего правого углов ограничивающего прямоугольника) через пробел.
 - 9) вывод итоговых координат верхнего левого и нижнего правого углов ограничивающего прямоугольника.
- Код программы представлен на рисунке 12.

```

#include <iostream>

int main() {
    int h, w;
    if (!(std::cin >> h >> w)) {
        return 0;
    }

    int min_row = h, max_row = -1;
    int min_col = w, max_col = -1;

    for (int r = 0; r < h; ++r) {
        for (int c = 0; c < w; ++c) {
            int val;
            std::cin >> val;
            if (val == 1) {
                if (r < min_row) min_row = r;
                if (r > max_row) max_row = r;
                if (c < min_col) min_col = c;
                if (c > max_col) max_col = c;
            }
        }
    }

    int top_left_row = min_row - 1;
    int top_left_col = min_col - 1;
    int bottom_right_row = max_row + 1;
    int bottom_right_col = max_col + 1;

    std::cout << top_left_row << " " << top_left_col << " "
                << bottom_right_row << " " << bottom_right_col << "\n";
    return 0;
}

```

Рисунок 12

Задание №6

В школьном кружке робототехники есть два вида микроконтроллеров (условно тип А и тип В) и два вида модулей управления мотором (условно тип 1 и тип 2). Выяснилось, что контроллер типа В и модуль управления типа 2 несовместимы. Использование микроконтроллеров и модулей управления в других комбинациях возможно. Имеется a микроконтроллеров типа А, b микроконтроллеров типа В, x модулей управления типа 1 и y модулей типа 2. Определите, какое максимальное число работающих пар из микроконтроллера и модуля управления мотором можно составить. Ваша программа должна ответить на n запросов.

Пример входных и выходных данных представлен на рисунке 13.

Sample Input:

```
3
5 2 3 7
10 2 12 2
3 8 5 6
```

Sample Output:

```
7 12 8
```

Рисунок 13

Алгоритм программы:

- 1) считывание количества запросов n и инициализация вектора res для сохранения результатов каждого из них;
- 2) итеративный обход всех n запросов со считыванием количества микроконтроллеров a , b и модулей управления x , y ;
- 3) первоочередная сборка пар для микроконтроллеров типа В, которые совместимы только с модулями типа 1: расчёт их максимального количества $max_pairs_B = std::min(b, x)$;
- 4) вычитание использованных элементов ($b -= max_pairs_B$, $x -= max_pairs_B$) с прибавлением полученного количества пар к итоговой сумме sum_pairs ;
- 5) добавление к sum_pairs количества пар, сформированных из универсальных микроконтроллеров типа А и всех оставшихся модулей по формуле $std::min(x + y, a)$, с последующим сохранением значения в вектор res ;
- 6) последовательный вывод всех элементов вектора res через пробел в качестве ответа на каждый запрос.

Код программы представлен на рисунке 14.

```

1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  int main() {
6      int n, a, b, x, y;
7      std::cin >> n;
8
9      std::vector<int> res;
10
11     // Максимальное количество всех пар и пар с микроконтроллерами типа В
12     int sum_pairs, max_pairs_B;
13
14     for (int i = 0; i < n; ++i) {
15         sum_pairs = max_pairs_B = 0;
16         std::cin >> a >> b >> x >> y;
17
18         max_pairs_B = std::min(b, x); // Подсчёт числа пар с микроконтроллерами типа В
19         b -= max_pairs_B;             // Вычет микроконтроллеров (В), уже использованных
20         x -= max_pairs_B;             // Вычет модулей управления (1), уже использованных
21         sum_pairs += max_pairs_B;     // Прибавка к общей сумме
22
23         // Формируем пары из оставшихся модулей контроля и микроконтроллеров типа В
24         sum_pairs += std::min(x + y, a);
25
26         res.push_back(sum_pairs);
27     }
28
29     for (int i = 0; i < n; ++i) {
30         std::cout << res[i] << " ";
31     }
32
33     std::cout << "\n";
34 }

```

Рисунок 12

Задание №7

На компьютере работника автосервиса нашли файл с последовательностью автомобильных номеров, обслуживавшихся в этом автосервисе. Так как файл был поврежден, некоторые данные отображаются неверно. Нужно определить, какие из них остались невредимыми.

Автомобильным номером является строка из шести символов. Первый символ – заглавная латинская буква, далее следует 3 цифры, и после – две заглавные латинские буквы. Например, строка "P142EQ" является номером. Вам будет дана строка, состоящая из шести символов, необходимо ответить, является ли строка автомобильным номером.

Пример входных и выходных данных представлен на рисунке 15.

Sample Input 1:

K040LE

Sample Output 1:

Yes

Sample Input 2:

M3239L

Sample Output 2:

No

Рисунок 15

Алгоритм программы:

- 1) считывание входной строки `number`, представляющей проверяемый автомобильный номер;
- 2) инициализация логического флага `flag = true` для отслеживания соответствия строки заданному шаблону;
- 3) итеративный посимвольный обход строки от индекса 0 до 5 с извлечением текущего символа `ch`;
- 4) проверка позиций с индексами от 1 до 3 (где должны располагаться цифры): если символ `ch` не входит в диапазон от '0' до '9', значение `flag` меняется на `false`;
- 5) проверка остальных позиций с индексами 0, 4 и 5 (где должны располагаться буквы): если символ `ch` не входит в диапазон от 'A' до 'Z', значение `flag` меняется на `false`;
- 6) вывод "Yes", если значение `flag` осталось `true`, или "No", если формат строки был нарушен.

Код программы представлен на рисунке 16.

```

1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string number;
6      std::cin >> number;
7
8      bool flag = true; //флаг "номер корректный"
9      for (int i = 0; i < 6; ++i) {
10         char ch = number[i];
11         //Проверяем места, где должны быть цифры
12         if (i ≥ 1 && i ≤ 3) {
13             if (!(ch ≤ '9' && ch ≥ '0')) {
14                 flag = false;
15             }
16             //Проверяем места, где должны быть буквы
17         } else {
18             if (!(ch ≤ 'Z' && ch ≥ 'A')) {
19                 flag = false;
20             }
21         }
22     }
23     if (flag) {
24         std::cout << "Yes";
25     } else {
26         std::cout << "No";
27     }
28 }

```

Рисунок 16

Задание №8

Составить светодиодную матрицу размером не менее 8 на 8 светодиодов (пример на рисунке ниже размером 4 на 4). На матрицу вывести инфографику с различными динамично меняющимися изображениями.

Алгоритм программы:

- 1) подключение библиотеки `Adafruit_NeoPixel` и инициализация объекта матрицы `pixels` на 64 светодиода с указанием управляющего пина микроконтроллера и начальной настройкой яркости в функции `setup()`;
- 2) реализация функции `getLedIndex(x, y)` для преобразования двумерных координат в одномерный индекс массива светодиодов. Функция включает проверку выхода за границы матрицы (с возвратом -1) и учитывает зигзагообразное (змейкой) подключение светодиодов, меняя формулу расчета индекса в зависимости от четности строки;
- 3) создание вспомогательных функций для отрисовки: `drawPixel` для установки заданного цвета конкретному пикселю по его координатам и `drawFrame` для вывода целого кадра, представленного матрицей 8x8, где цвет применяется только к ячейкам со значением 1;
- 4) инициализация набора двумерных массивов `frame1` — `frame5` и `heart`, хранящих бинарные маски для воспроизведения различных паттернов (кадров анимации расширяющегося квадрата и контура сердца);
- 5) последовательное выполнение трех динамических анимаций в основном цикле `loop()`: поочередный вывод кадров расширяющегося квадрата через `drawFrame` с временными задержками `delay()`, анимация «Змейка» с последовательным зажиганием и гашением каждого светодиода по его индексу, а также пульсирующее циклическое переключение цветов для кадра «Сердце».

Код программы представлен на рисунках 17-19. Пример работы программы показан на рисунке 20.

```

1  #include <Adafruit_NeoPixel.h>
2
3  #define PIN 3
4  #define NUMPIXELS 64
5
6  Adafruit_NeoPixel pixels(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);
7
8  void setup() {
9      pixels.begin();
10     pixels.setBrightness(100);
11     pixels.show();
12 }
13
14 // Функция для получения индекса светодиода по его координатам
15 int getLedIndex(int x, int y) {
16     if (x < 0 || x > 7 || y < 0 || y > 7) return -1;
17     if (y % 2 == 0) {
18         return (y * 8) + 7 - x;
19     } else {
20         return (y * 8) + x;
21     }
22 }
23
24 // Функция-обертка для рисования
25 void drawPixel(int x, int y, uint32_t color) {
26     int index = getLedIndex(x, y);
27     if (index != -1) {
28         pixels.setPixelColor(index, color);
29     }
30 }
31
32 // Функция для отрисовки картинки на матрице
33 void drawFrame(uint8_t frame[8][8], uint32_t color) {
34     pixels.clear();
35     for(int y = 0; y < 8; y++) {
36         for(int x = 0; x < 8; x++) {
37             if (frame[y][x] == 1) {
38                 drawPixel(x, y, color); // Зажигаем только те, где прописана единица
39             }
40         }
41     }
42     pixels.show(); // Отправляем данные на матрицу
43 }
44
45 uint8_t frame1[8][8] = {
46     {0,0,0,0,0,0,0,0},
47     {0,0,0,0,0,0,0,0},
48     {0,0,0,0,0,0,0,0},
49     {0,0,0,1,1,0,0,0},
50     {0,0,0,1,1,0,0,0},
51     {0,0,0,0,0,0,0,0},
52     {0,0,0,0,0,0,0,0},
53     {0,0,0,0,0,0,0,0}
54 };
55
56 uint8_t frame2[8][8] = {
57     {0,0,0,0,0,0,0,0},
58     {0,0,0,0,0,0,0,0},
59     {0,0,1,1,1,1,0,0},
60     {0,0,1,0,0,1,0,0},
61     {0,0,1,0,0,1,0,0},
62     {0,0,1,1,1,1,0,0},
63     {0,0,0,0,0,0,0,0},
64     {0,0,0,0,0,0,0,0}
65 };

```

Рисунок 17

```

67  uint8_t frame3[8][8] = {
68      {0,0,0,0,0,0,0,0},
69      {0,1,1,1,1,1,1,0},
70      {0,1,0,0,0,0,1,0},
71      {0,1,0,0,0,0,1,0},
72      {0,1,0,0,0,0,1,0},
73      {0,1,0,0,0,0,1,0},
74      {0,1,1,1,1,1,1,0},
75      {0,0,0,0,0,0,0,0}
76  };
77
78  uint8_t frame4[8][8] = {
79      {1,1,1,1,1,1,1,1},
80      {1,0,0,0,0,0,0,1},
81      {1,0,0,0,0,0,0,1},
82      {1,0,0,0,0,0,0,1},
83      {1,0,0,0,0,0,0,1},
84      {1,0,0,0,0,0,0,1},
85      {1,0,0,0,0,0,0,1},
86      {1,1,1,1,1,1,1,1}
87  };
88
89  uint8_t frame5[8][8] = {
90      {1,1,1,1,1,1,1,1},
91      {1,1,1,1,1,1,1,1},
92      {1,1,1,1,1,1,1,1},
93      {1,1,1,1,1,1,1,1},
94      {1,1,1,1,1,1,1,1},
95      {1,1,1,1,1,1,1,1},
96      {1,1,1,1,1,1,1,1},
97      {1,1,1,1,1,1,1,1}
98  };
99
100 uint8_t heart[8][8] = {
101     {0,0,0,0,0,0,0,0},
102     {0,0,1,0,0,1,0,0},
103     {0,1,1,1,1,1,1,0},
104     {0,1,1,1,1,1,1,0},
105     {0,0,1,1,1,1,0,0},
106     {0,0,0,1,1,0,0,0},
107     {0,0,0,0,0,0,0,0},
108     {0,0,0,0,0,0,0,0}
109 };
110

```

Рисунок 18

```

111 void loop() {
112     //1. Анимация "Расходящиеся квадраты"
113     drawFrame(frame1, pixels.Color(0, 0, 255));
114     delay(500);
115
116     drawFrame(frame2, pixels.Color(0, 0, 255));
117     delay(500);
118
119     drawFrame(frame3, pixels.Color(0, 0, 255));
120     delay(500);
121
122     drawFrame(frame4, pixels.Color(0, 0, 255));
123     delay(500);
124
125     drawFrame(frame5, pixels.Color(0, 0, 255));
126     delay(500);
127
128     // 2. Анимация "Змейка"
129     // По очереди зажигаем светодиоды зеленым цветом
130     for(int i = 0; i < NUMPIXELS; i++) {
131         pixels.setPixelColor(i, pixels.Color(0, 255, 0));
132         pixels.show();
133         delay(30);
134     }
135
136     delay(500); // Небольшая пауза
137
138     // Так же по очереди их гасим
139     for(int i = 0; i < NUMPIXELS; i++) {
140         pixels.setPixelColor(i, pixels.Color(0, 0, 0)); // Выключить цвет
141         pixels.show();
142         delay(30);
143     }
144
145     delay(500);
146
147     //3. Анимация "Сердце"
148
149     for (int i = 0; i < 10; ++i) {
150         drawFrame(heart, pixels.Color(255, 0, 0));
151         delay(500);
152         drawFrame(heart, pixels.Color(125, 0, 0));
153         delay(500);
154     }
155 }

```

Рисунок 19

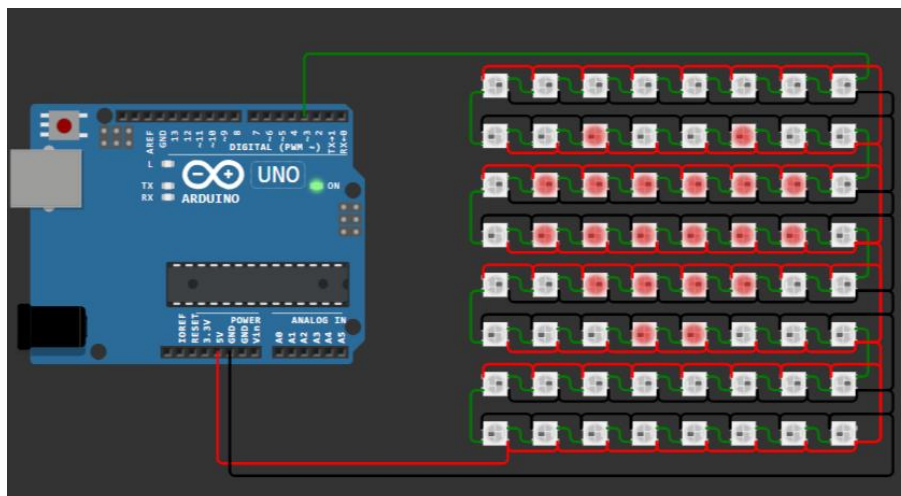


Рисунок 20

Задание №9

Задачи:

- 1) собрать схему имитирующую работу автоматических дверей
- 2) подобрать номинал резисторов для светодиодов
- 3) написать программу для управления процессом работы автоматических дверей.

Схема приведена на рисунке 21.

Зеленый светодиод – двери открываются.

Красный светодиод – двери закрываются.

Фоторезистор имитируют процесс приближения-удаления человека от дверей.

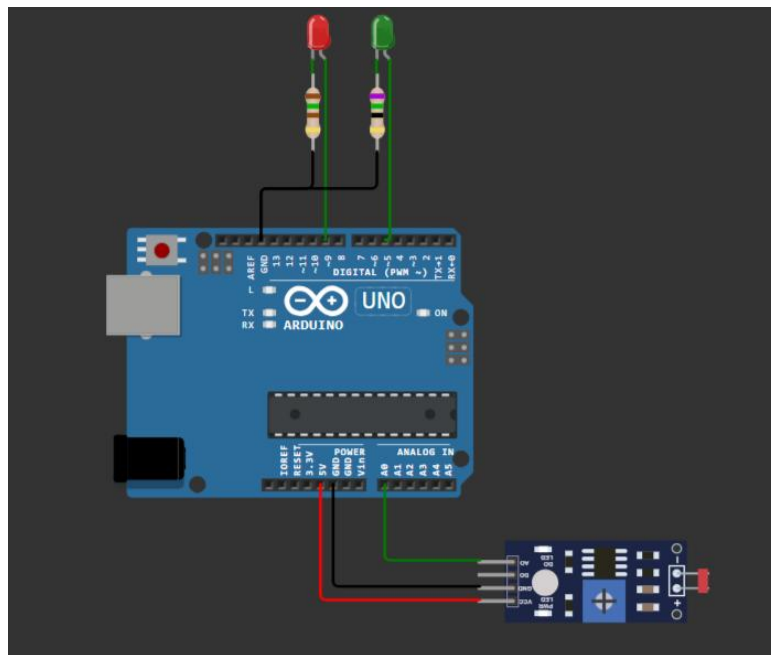


Рисунок 21 — Схема управления работой автоматических дверей

Алгоритм программы:

- 1) инициализация констант пинов подключения компонентов (redLedPin = 9, greenLedPin = 5, sensorPin = A0) и логического флага isDoorOpen со значением false для отслеживания текущего состояния дверей;
- 2) настройка пинов светодиодов на вывод (OUTPUT), запуск передачи данных через последовательный порт (Serial.begin(9600)) и установка начального состояния схемы (красный светодиод горит, зелёный выключен) в функции setup();
- 3) непрерывное считывание показаний аналогового сигнала с фоторезистора sensorValue = analogRead(sensorPin) в цикле loop();
- 4) проверка приближения человека (sensorValue > 512): если двери закрыты, выключается красный светодиод, включается зелёный, в

последовательный порт выводится сообщение «Двери открываются!», значение isDoorOpen меняется на true и выполняется задержка 3 секунды;

5) обработка противоположного условия (sensorValue <= 512): если двери открыты (isDoorOpen), гаснет зелёный светодиод, включается красный, в порт выводится сообщение «Двери закрываются!», а флаг isDoorOpen возвращается в значение false;

6) выполнение небольшой задержки delay(100) в конце каждого шага цикла loop() для стабилизации считывания показаний датчика.

Код программы представлен на рисунке 23. Пример работы программы показан на рисунке 24.

```
1  const int redLedPin = 9;
2  const int greenLedPin = 5;
3  const int sensorPin = A0;
4
5  bool isDoorOpen = false;
6
7  void setup() {
8      pinMode(redLedPin, OUTPUT);
9      pinMode(greenLedPin, OUTPUT);
10
11     // Инициализация последовательного порта для вывода сообщений
12     Serial.begin(9600);
13
14     // По умолчанию горит красный светодиод (дверь закрыта)
15     digitalWrite(redLedPin, HIGH);
16     digitalWrite(greenLedPin, LOW);
17 }
18
19 void loop() {
20     // Чтение значения с фоторезистора
21     int sensorValue = analogRead(sensorPin);
22
23     if (sensorValue > 512) {
24         if (!isDoorOpen) {
25             digitalWrite(redLedPin, LOW);
26             digitalWrite(greenLedPin, HIGH);
27             Serial.println("Двери открываются!");
28
29             isDoorOpen = true;
30
31             delay(3000); //Перерыв 3 секунды после открытия двери
32         }
33     } else {
34         if (isDoorOpen) {
35             digitalWrite(greenLedPin, LOW);
36             digitalWrite(redLedPin, HIGH);
37             Serial.println("Двери закрываются!");
38
39             isDoorOpen = false;
40         }
41     }
42
43     delay(100);
44 }
```

Рисунок 23

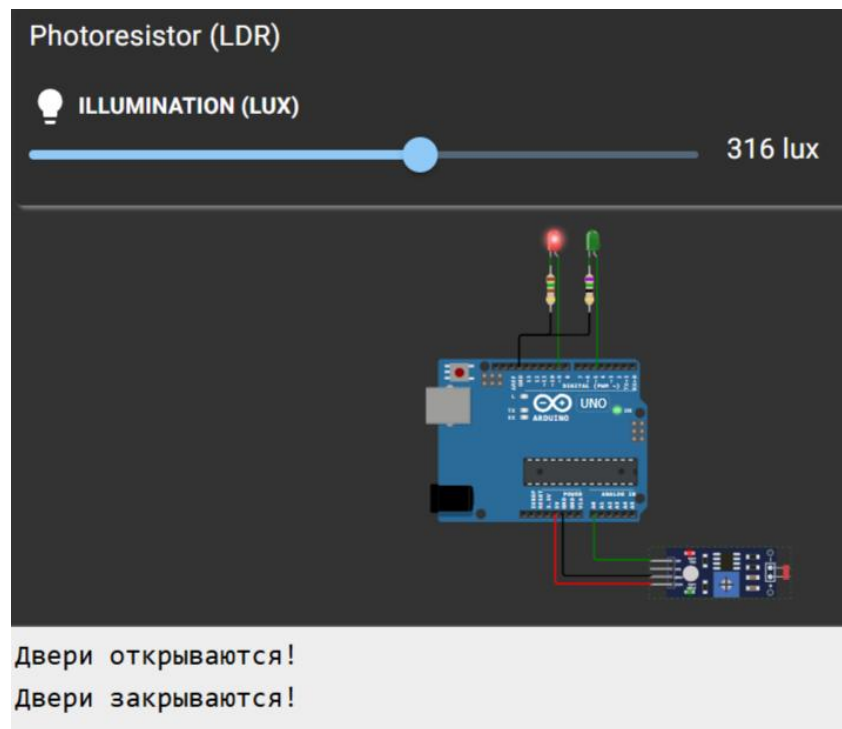


Рисунок 24

Задание №10

Задачи:

- 1) собрать схему подключения сервопривода;
- 2) написать программу для управления сервоприводом через последовательный порт.

Логика работы программы:

- 1) программа находится в ожидании ввода данных в последовательный порт;
- 2) когда в последовательный порт вводятся данные, программа проверяет их на корректность (Должны приниматься только числовые значения в диапазоне от 0 до 180);
- 3) если введен угол поворота, отличный от текущего угла, сервопривод плавно поворачивается в заданное положение.

Алгоритм программы:

- 1) подключение библиотеки `<Servo.h>`, определение управляющего пина `SERVO_PIN = 3`, создание объекта сервопривода `myServo` и переменной `currentAngle = 0` для хранения текущего угла поворота;
- 2) запуск передачи данных через последовательный порт (`Serial.begin(9600)`), привязка сервопривода к управляющему пину (`myServo.attach(SERVO_PIN)`), установка начального положения (0 градусов) и вывод текстового приглашения к вводу в функции `setup()`;
- 3) проверка наличия поступивших данных в последовательном порту (`Serial.available() > 0`) в цикле `loop()`, считывание введенной строки `inputString` до символа переноса строки `\n` и удаление крайних пробелов методом `trim()`;
- 4) проверка корректности структуры строки через вспомогательную функцию `isValidNumber()`: если строка не пуста и состоит исключительно из цифр (`isDigit()`), она преобразуется в числовое значение угла `targetAngle = inputString.toInt()`, иначе выводится сообщение об ошибке некорректных данных;
- 5) проверка диапазона заданного угла (`targetAngle >= 0 && targetAngle <= 180`): при выходе за пределы выводится ошибка диапазона; при совпадении с текущим значением (`targetAngle == currentAngle`) сообщается, что привод уже находится в данном положении;
- 6) вызов функции `moveServoSmoothly(currentAngle, targetAngle)` для плавного поворота сервопривода при успешной валидации, обновление значения `currentAngle = targetAngle` и вывод сообщения с новым положением в последовательный порт;
- 7) реализация плавного перемещения в функции `moveServoSmoothly()` с помощью цикла `for`: пошаговое изменение угла на 1 градус (увеличение или уменьшение) с вызовом `myServo.write(pos)` и задержкой между шагами.

Код программы представлен на рисунках 25-26. Пример работы программы показан на рисунке 27.

```

1  #include <Servo.h>
2
3  #define SERVO_PIN 3
4
5  Servo myServo; //Создаём объект сервопривода
6
7  // Переменная для хранения текущего угла
8  int currentAngle = 0;
9
10 void setup() {
11     // Инициализируем последовательный порт
12     Serial.begin(9600);
13
14     // Подключаем сервопривод к пину
15     myServo.attach(SERVO_PIN);
16
17     // Устанавливаем в начальное положение
18     myServo.write(currentAngle);
19     Serial.println("Введите угол от 0 до 180 градусов:");
20 }
21
22 void loop() {
23     // Ожидание ввода данных в последовательный порт
24     if (Serial.available() > 0) {
25
26         // Чтение строки
27         String inputString = Serial.readStringUntil('\n');
28
29         // Удаление случайных пробелов
30         inputString.trim();
31
32         // Проверяем строку на корректность
33         if (isValidNumber(inputString)) {
34             int targetAngle = inputString.toInt();
35             if (targetAngle ≥ 0 && targetAngle ≤ 180) {
36                 if (targetAngle ≠ currentAngle) {
37                     Serial.print("Перемещаю сервопривод в положение: ");
38                     Serial.print(targetAngle);
39                     Serial.println(" град.");
40
41                     moveServoSmoothly(currentAngle, targetAngle);
42
43                     // Запоминаем новое текущее положение
44                     currentAngle = targetAngle;
45                 } else {
46                     Serial.println("Сервопривод уже находится в этом положении.");
47                 }
48             } else {
49                 Serial.println("Ошибка! Диапазон угла должен быть строго от 0 до 180.");
50             }
51         } else {
52             Serial.println("Ошибка! Введены некорректные данные. Допускаются только числа.");
53         }
54     }
55 }
56
57 }

```

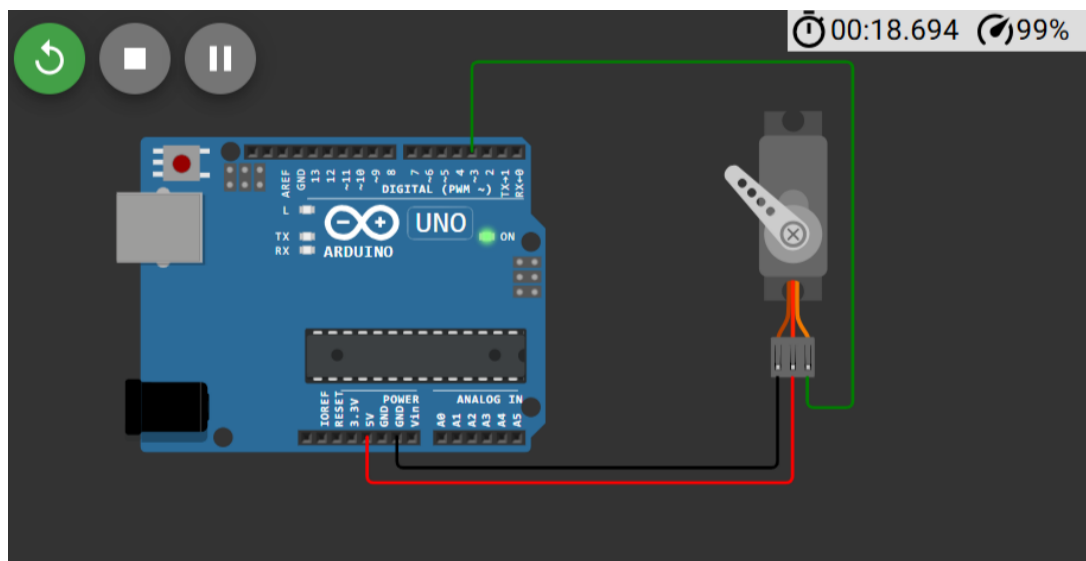
Рисунок 25

```

59 // Проверка на то, что строка состоит только из цифр
60 bool isValidNumber(String& str) {
61     if (str.length() == 0) {
62         return false;
63     }
64
65     for (byte i = 0; i < str.length(); i++) {
66         if (!isDigit(str[i])) {
67             return false;
68         }
69     }
70     return true;
71 }
72
73 // Функция для плавного перемещения
74 void moveServoSmoothly(int& fromAngle, int& toAngle) {
75     if (fromAngle < toAngle) {
76         // Если новый угол больше текущего – плавно увеличиваем
77         for (int pos = fromAngle; pos ≤ toAngle; pos++) {
78             myServo.write(pos);
79             delay(20);
80         }
81     } else {
82         // Если новый угол меньше текущего – плавно уменьшаем
83         for (int pos = fromAngle; pos ≥ toAngle; pos--) {
84             myServo.write(pos);
85             delay(20);
86         }
87     }
88 }

```

Рисунок 26



Введите угол от 0 до 180 градусов:
 Перемещая сервопривод в положение: 45 град.

Рисунок 27

Задание №11

Вариант №21.

Найдите самый крупный объект на изображении. Определите центр этого объекта. Обведите самый крупный объект красной рамкой. 28.

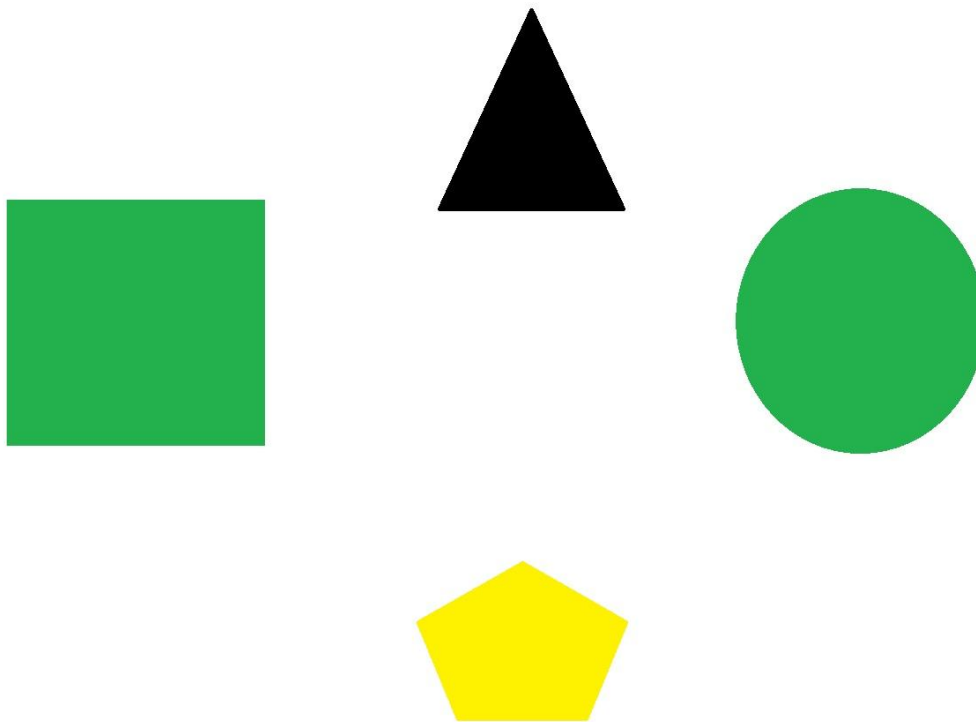


Рисунок 28

Алгоритм программы:

- 1) Загрузка изображения;
- 2) Преобразование в оттенки серого
- 3) Бинаризация изображения
- 4) Поиск контуров всех объектов
- 5) Поиск контура с максимальной площадью
- 6) Расчет центра (центроида) с помощью моментов
- 7) Рисование красной рамки вокруг крупнейшего объекта
- 8) Рисование точки и текста с координатами центра

Код программы представлен на рисунке 29. Пример работы программы показан на рисунке 30.

```

import cv2

image_path = "img.png" # Укажите путь к вашему изображению
img = cv2.imread(image_path)
output = img.copy()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, thresh: 240, maxval: 255, cv2.THRESH_BINARY_INV)
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

if contours:
    largest_contour = max(contours, key=cv2.contourArea)
    M = cv2.moments(largest_contour)
    if M["m00"] != 0:
        cX = int(M["m10"] / M["m00"])
        cY = int(M["m01"] / M["m00"])
    else:
        cX, cY = 0, 0
    x, y, w, h = cv2.boundingRect(largest_contour)
    cv2.rectangle(output, (x, y), (x + w, y + h), (0, 0, 255), 3)
    cv2.circle(output, center: (cX, cY), radius: 5, color: (0, 0, 255), -1)
    print(f"Самый крупный объект найден:")
    print(f"Площадь: {cv2.contourArea(largest_contour)}px")
    print(f"Центр объекта: X={cX}, Y={cY}")
    cv2.imwrite(filename: "result.png", output)
    cv2.imshow(winname: "Largest Object", output)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    print("Объекты на изображении не найдены.")

```

Рисунок 29

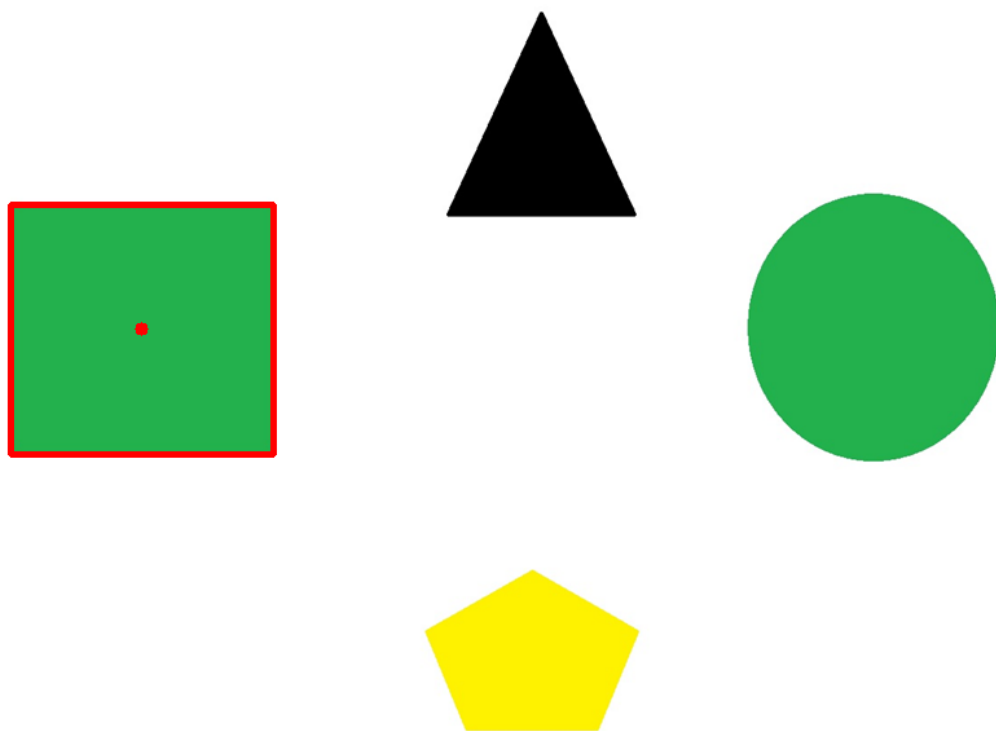


Рисунок 30

Заключение

В процессе прохождения учебной практики были успешно выполнены все 11 практических заданий. Были реализованы и отлажены алгоритмы на языке C++, созданы действующие микроконтроллерные схемы на базе Arduino (со светодиодными матрицами, аналоговыми датчиками и сервоприводом), а также разработана программа цветовой сегментации изображений в OpenCV.

Все созданные программы продемонстрировали корректную работу на контрольных наборах данных и схемах. Выполненные задания позволили закрепить теоретический материал и получить ценный практический опыт в сфере алгоритмизации, микроконтроллерной техники и обработки визуальной информации.

Список литературы

1. Wokwi Arduino Simulator: онлайн-симулятор Arduino-проектов [Электронный ресурс]. – URL: <https://wokwi.com/arduino> (дата обращения: 22.08.2026).
2. Подробное руководство по работе со светодиодными лентами WS2812B [Электронный ресурс]. – URL: https://alexgyver.ru/ws2812_guide (дата обращения: 22.08.2026).
3. Руководство по работе со светодиодными матрицами [Электронный ресурс]. – URL: https://alexgyver.ru/matrix_guide (дата обращения: 22.08.2026).
4. Работа с аналоговыми входами Arduino [Электронный ресурс]. – URL: <https://alexgyver.ru/lessons/analog-pins> (дата обращения: 22.08.2026).
5. Работа с последовательным портом Serial [Электронный ресурс]. – URL: <https://alexgyver.ru/lessons/serial> (дата обращения: 22.08.2026).
6. Подключение и управление сервоприводами [Электронный ресурс]. – URL: <https://alexgyver.ru/lessons/servo> (дата обращения: 22.08.2026).